- Contexte
- Formalismes et notation
- Spécification formelle et systèmes
  - machines à état
  - réseaux de Petri
- spécification formelles des ppts
  - Logique temporelle LTL
  - Logique arborescente CTL
  - Model Checking LTL
  - _____ CTL
  - conclusion

Lundi 18 septembre 2023
Systèmes de Transitions et Model Checking
CM.

Kais Klai
B216
klai@lipn.fr

Introduction à la vérification formelle
↳ précis, rigoureux

Processus P

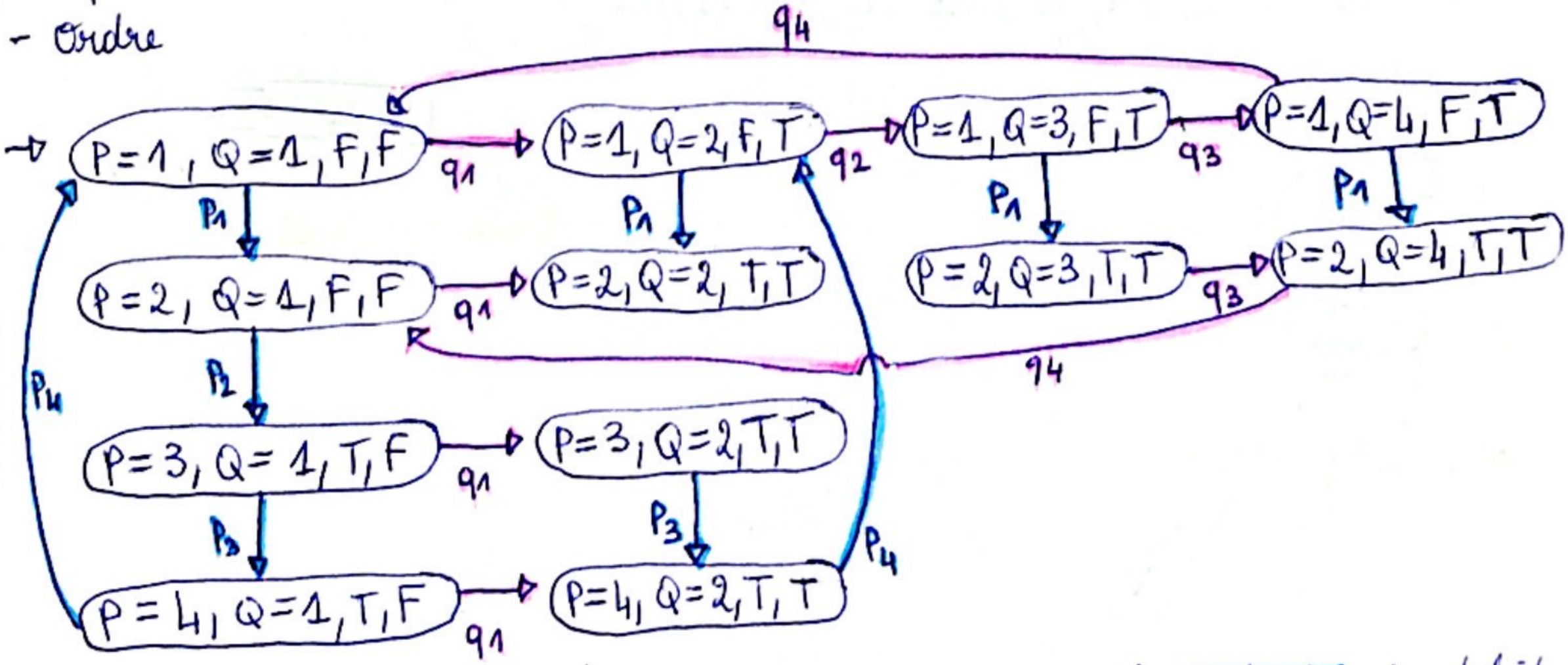
while (True)
P_1 ..... req p ← True
P_2 ..... wait (req Q = False)
P_3 ..... section critique ——▷ objet qui ne permet pas le partage
P_4 ..... req p ← False

Processus Q

while (True)
Q_1 ..... req Q ← True
Q_2 ..... wait (req p = False)
Q_3 ..... section critique
Q_4 ..... req Q ← False

Initialement, req p = req Q = False

- Exclusion mutuelle
- Equité
- Ordre



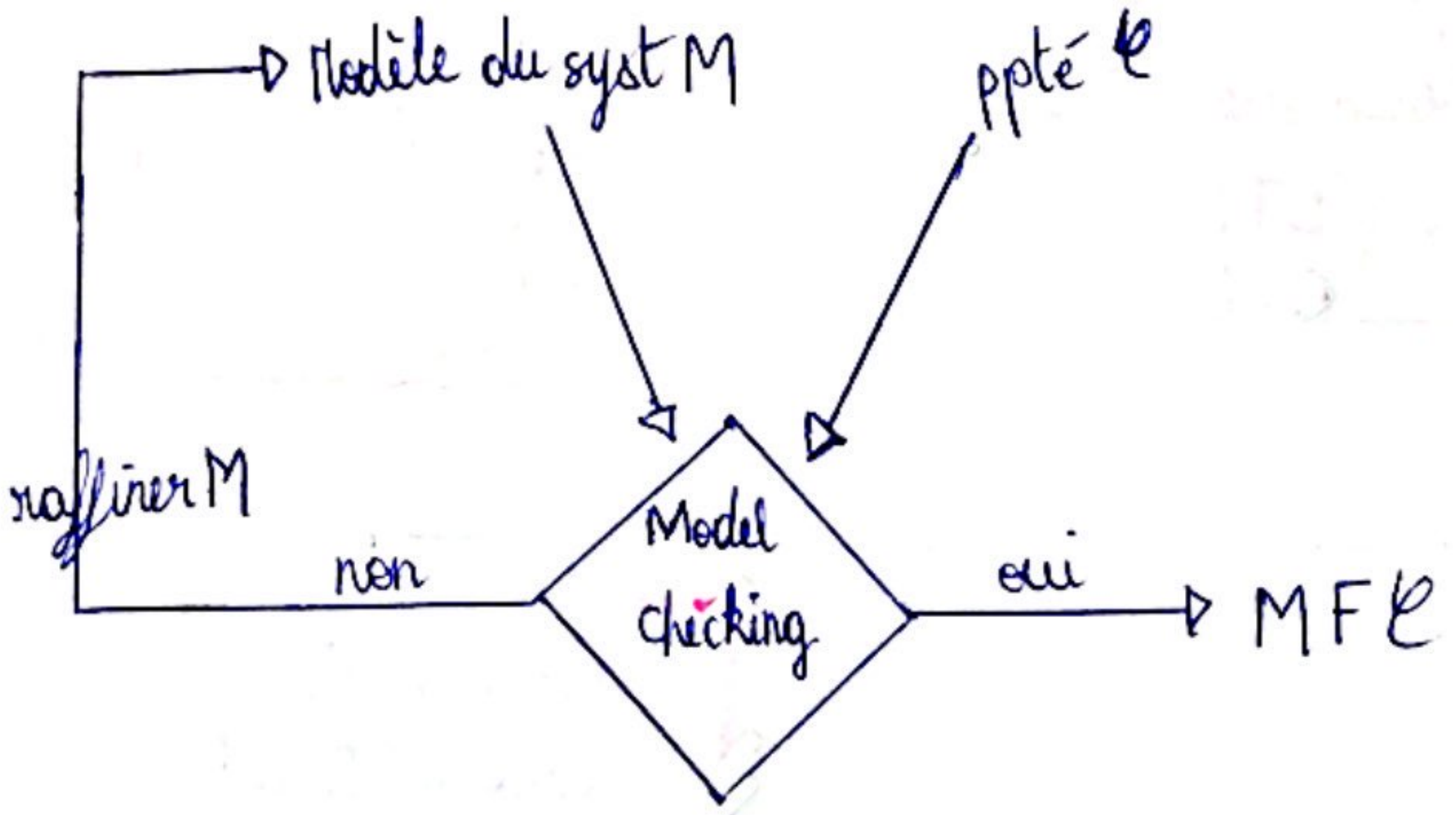
- Non, il ne peut pas accéder à l'état P=3, Q=3, T, T donc l'exclusion mutuelle est satisfaite. (Jamais P=3 et Q=3 ✓)
- A chaque fois qu'il finit en/ demande section critique, le processus P ou Q finit par l'avoir
  ↳ équité

Scanned with CamScanner

➡ Tous les chemins qui commencent par un état où p=2 passe par un état où p=3 et de même pour Q car par exemple, l'état de blocage ne vérifie pas ça.

Répondre à une requête dans un temps fini ⇒ équité

- **Pas en ordre :** Tout chemin commençant par un état p = 2 et q=1 atteint inévitablement un état p=3 sans jamais passer par un état q=3.



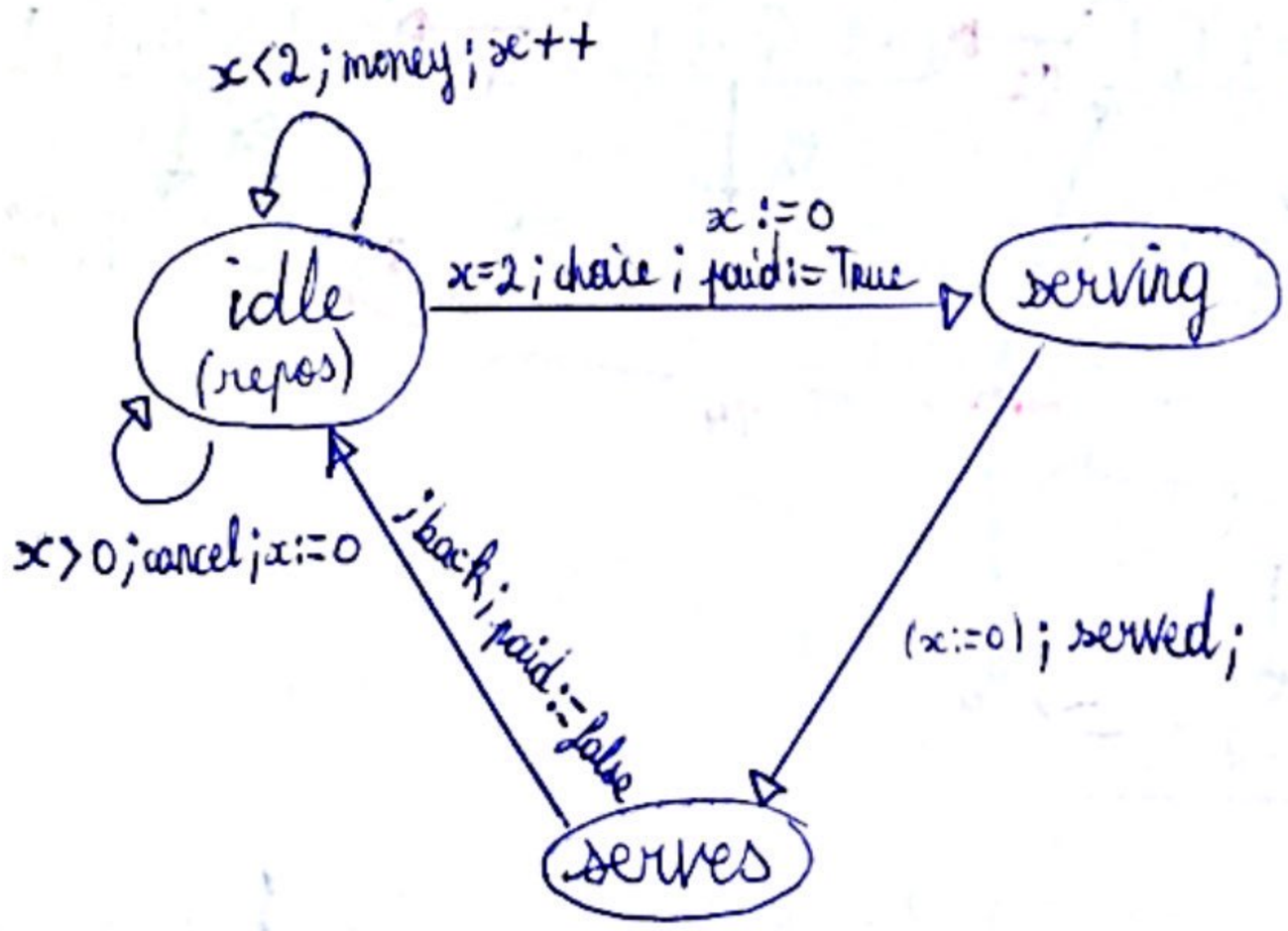
Machine à états

- syntaxe

une machine à états est un quadruplet (C, V, A, T) où

- C : un ens. fini d'états de contrôle
- V : un ens. de variables
- A : un ens. d'actions
- T : une relation de transitions

(source, précondition, action, postcondition, destination)



(machines d'états)

Scanned with CamScanner

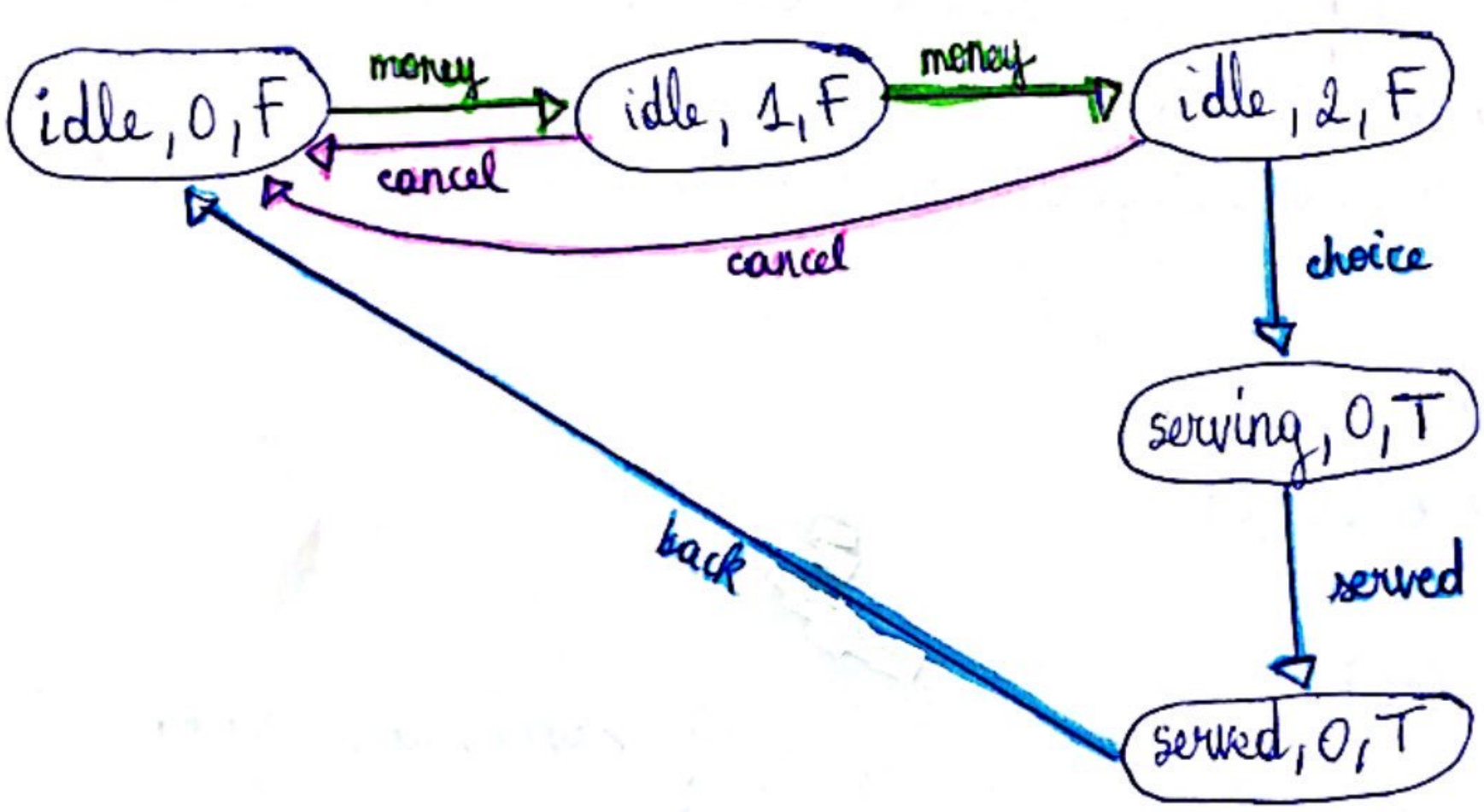
Sémantique : est représenté par un **système de transition** $(S, A, S_0, \rightarrow)$

- S : un ens. d'états
- A : ens. d'action
- S_0 : état initial

$\rightarrow \subseteq S \times A \times S$: relation de transition

un état = (état de contrôle, valeurs des variables)

(systèmes de transitions)



$(idle, 0, F) \xrightarrow{money} (idle, 1, F) \xrightarrow{cancel} (idle, 0, F) \xrightarrow{money} (idle, 1, F) \xrightarrow{money} (idle, 2, F) \xrightarrow{choice} \dots$

⇩

séquence, une exécution ou un mot

money, cancel, money, money, choice ...

↳ langage du système

Scanned with CamScanner

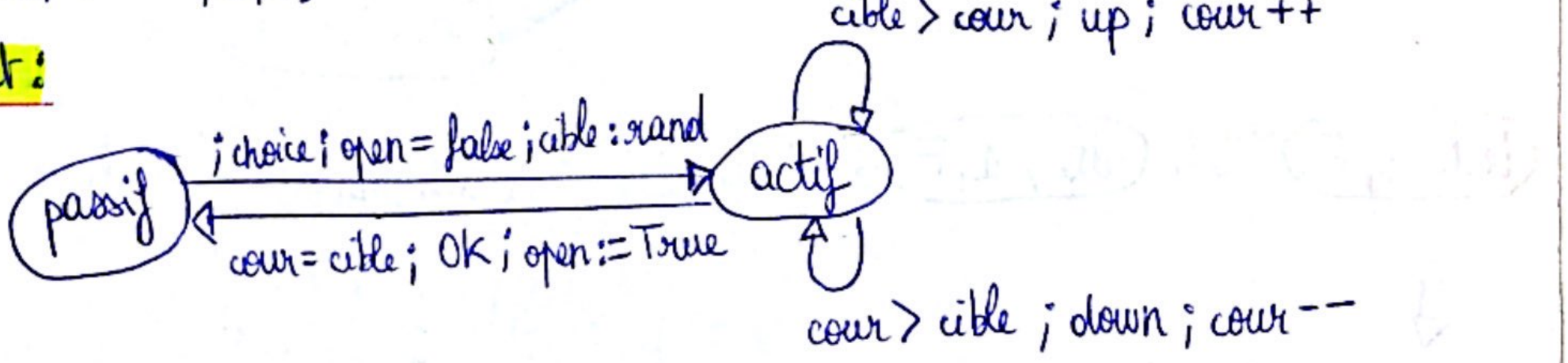
Modélisation d'un système de contrôle d'un ascenseur à 3 étages.

18/09/23
TD

- Le contrôleur garde en mémoire l'étage courant et l'étage cible.
- Le contrôleur a 2 modes : actif et passif.
- En mode actif, si l'étage cible est atteint, la porte de l'ascenseur s'ouvre et le contrôleur passe en mode passif.
- En mode actif, si l'étage cible est > à l'étage courant. Le contrôleur fait s'élever l'ascenseur.
- _____ < _____ descendre _____.
- En mode passif, une personne peut entrer dans l'ascenseur et choisit un état cible. La porte est alors fermée et le contrôleur passe en mode actif.
- Initialement, l'ascenseur est à l'étage 0 (porte est ouverte) et le contrôleur en mode passif.

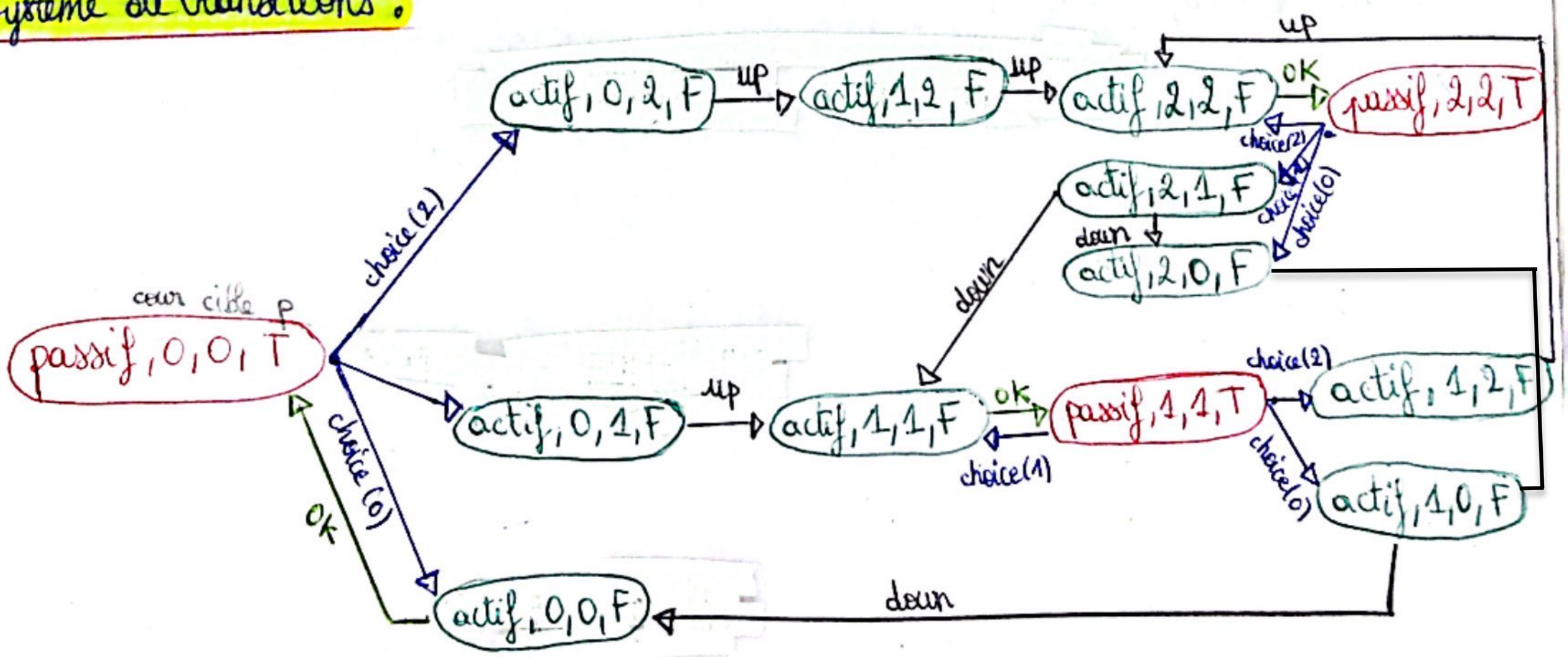
- C : {passif, actif}
- V : cour, cible : int ∈ [0,2]
  variable aléatoire ∈ [0,2]
  open : booléen
- A : {up, down, choice, OK}

Machine à état :



29/09/23

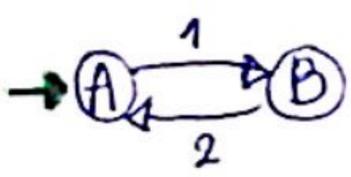
système de transitions :

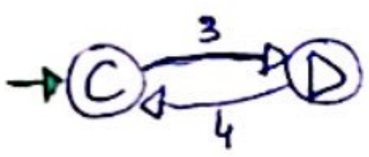


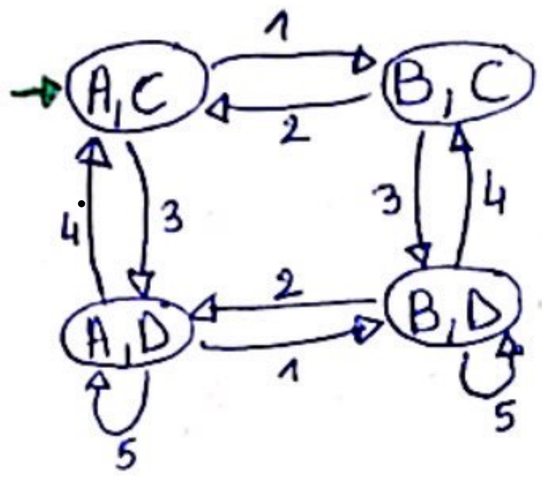
Scanned with CamScanner

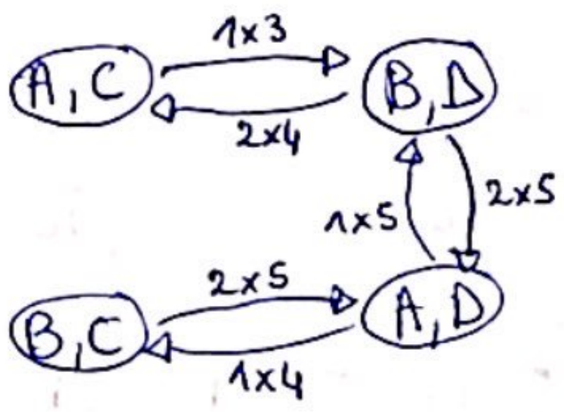
29/09/2023
CM+TD

composition : - asynchrone
- synchrone









$ST_1 = < S_1, s_{01}, A_1, \longrightarrow 1 >$

$ST_2 = < S_2, s_{02}, A_2, \longrightarrow 2 >$

$ST_1 \otimes ST_2 = ST = < S, s_0, A, \rightarrow >$

↑ produit synchronisé

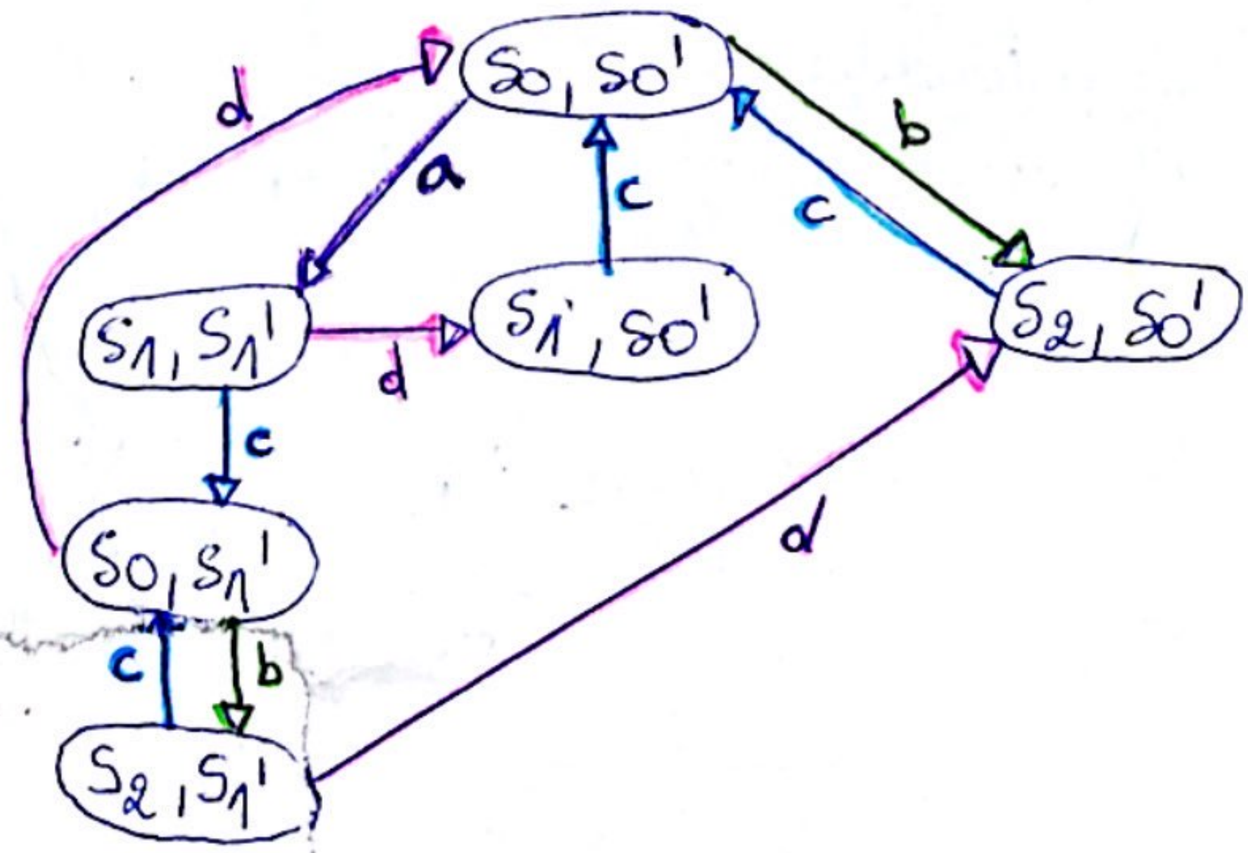
$$\begin{cases} S \subseteq S_1 \times S_2 \\ s_0 = (s_{01}, s_{02}) \\ A = A_1 \cup A_2 \\ \rightarrow \subseteq S \times A \times S \end{cases}$$

$(s_1, s_2) \xrightarrow{a} (s_1', s_2')$ ssi
① $a \in A_1 \setminus A_2 \Rightarrow s_1 \xrightarrow{a} s_1'$ et $s_2' = s_2$
② $a \in A_2 \setminus A_1 \Rightarrow s_1' = s_1$ et $s_2 \xrightarrow{a} s_2'$
③ $a \in A_1 \cap A_2 \Rightarrow s_1 \xrightarrow{a} s_1'$ et $s_2 \xrightarrow{a} s_2'$





construire produit synchronisé entre ces 2 sous-systèmes.



Scanned with CamScanner

Syntax :

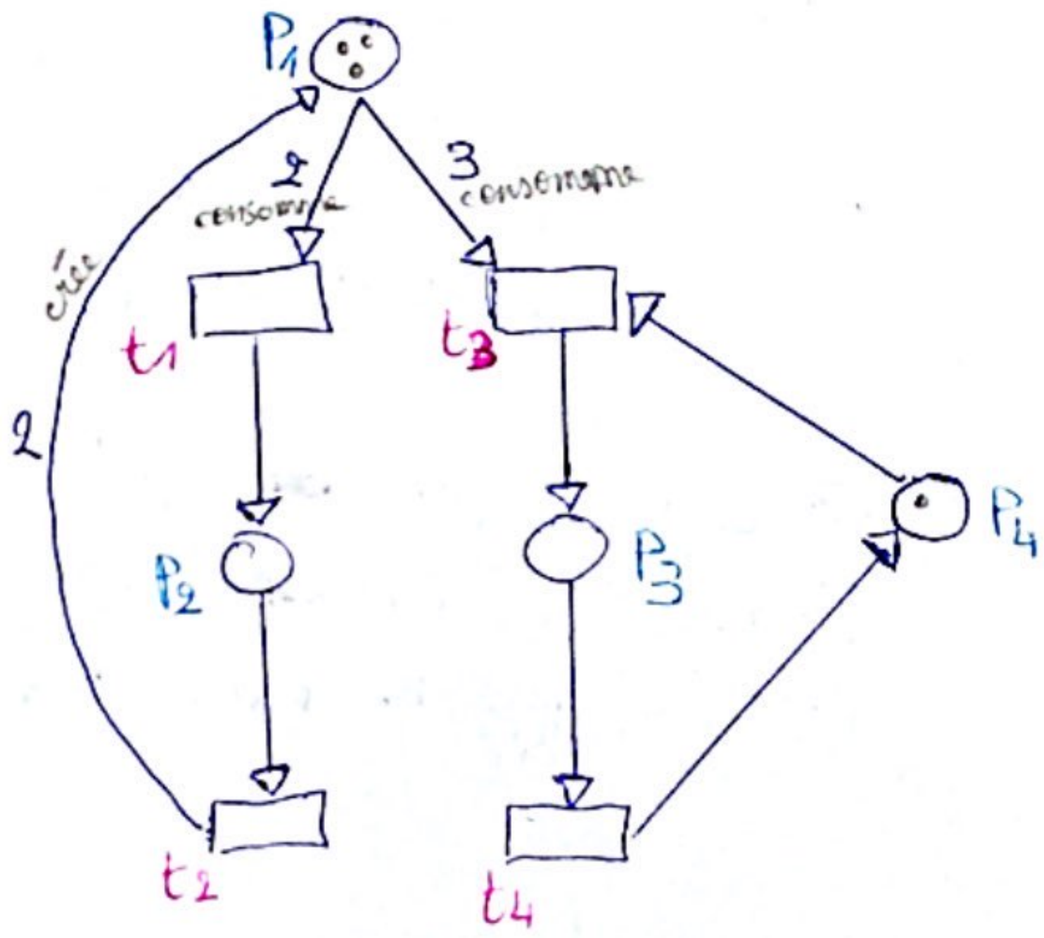
Réseaux de Petri

Un réseau de Petri est un quadruplet $R = \langle P, T, F, W \rangle$ tq :

- P est un ensemble fini de places (○)
- T _____ transitions (▭)
- $F \subseteq (P \times T) \cup (T \times P)$ est un ens. d'arcs.
- $W : F \to \mathbb{N}$ est une fct° associant à chaque arc un poids (nb entier)

Rq : $P \cap T = \emptyset$

$P \cup T \neq \emptyset$

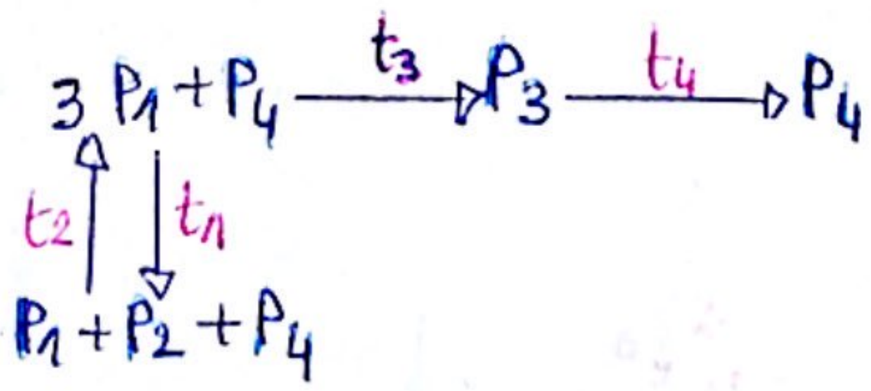


(si P_4 a rien (arc entrant) alors t_3 et t_1 infranchissables)

Pre : matrice des préconditions

Pos : _____ postconditions

C : _____ d'incidence



C = Post − Pre

arcs entrants dans t

Pre	t_1	t_2	t_3	t_4
P_1	2 (arc)	0	3	0
P_2	0	1	0	0
P_3	0	0	0	1
P_4	0	0	1	0

arcs sortants de t

Post	t_1	t_2	t_3	t_4
P_1	0	2	0	0
P_2	1	0	0	0
P_3	0	0	1	0
P_4	0	0	0	1

C (arc sort − arc entr)

	t_1	t_2	t_3	t_4
P_1	-2	2	-3	0
P_2	1	-1	0	0
P_3	0	0	1	-1
P_4	0	0	-1	1

marquage initial (état initial)

$m : P \to \mathbb{N}$

$\Rightarrow$

P_1	1
P_2	0
P_3	0
P_4	1

t_1 → (1, 0, 0, 1) → t_3 → (0, 0, 1, 0) → t_4 → (0, 0, 0, 1)

m + c(t)

Sémantique :

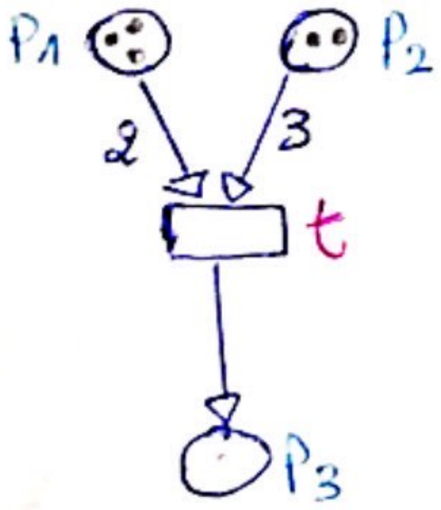
- Franchissabilité d'une transition

Une transition t est franchissable à partir d'un marquage m ssi :

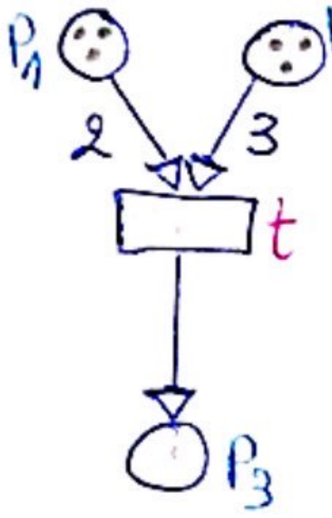
$$\forall p \in P,\ m(P) \geq Pre(Pt) \quad \text{càd} \quad m \geq Pre(t)$$

On note $m \xrightarrow{t}$

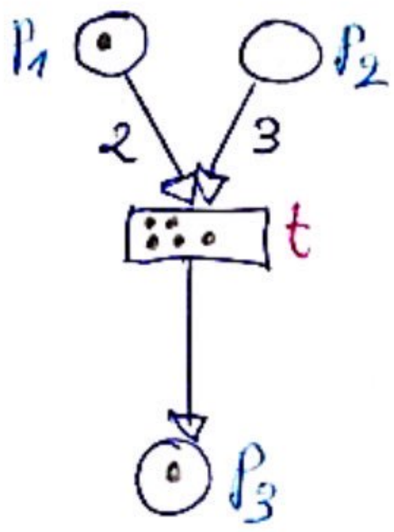
Franchissement si $m \xrightarrow{t}$ alors le franchissement de t à partir de m mène vers un marquage $m' = m - Pre(t) + Post(t) = m + c(t)$



Pas franchissable car P_2 a 2 pts mais l'arc en consomme 3.



Franchissable



avant

P_1	3
P_2	3
P_3	0

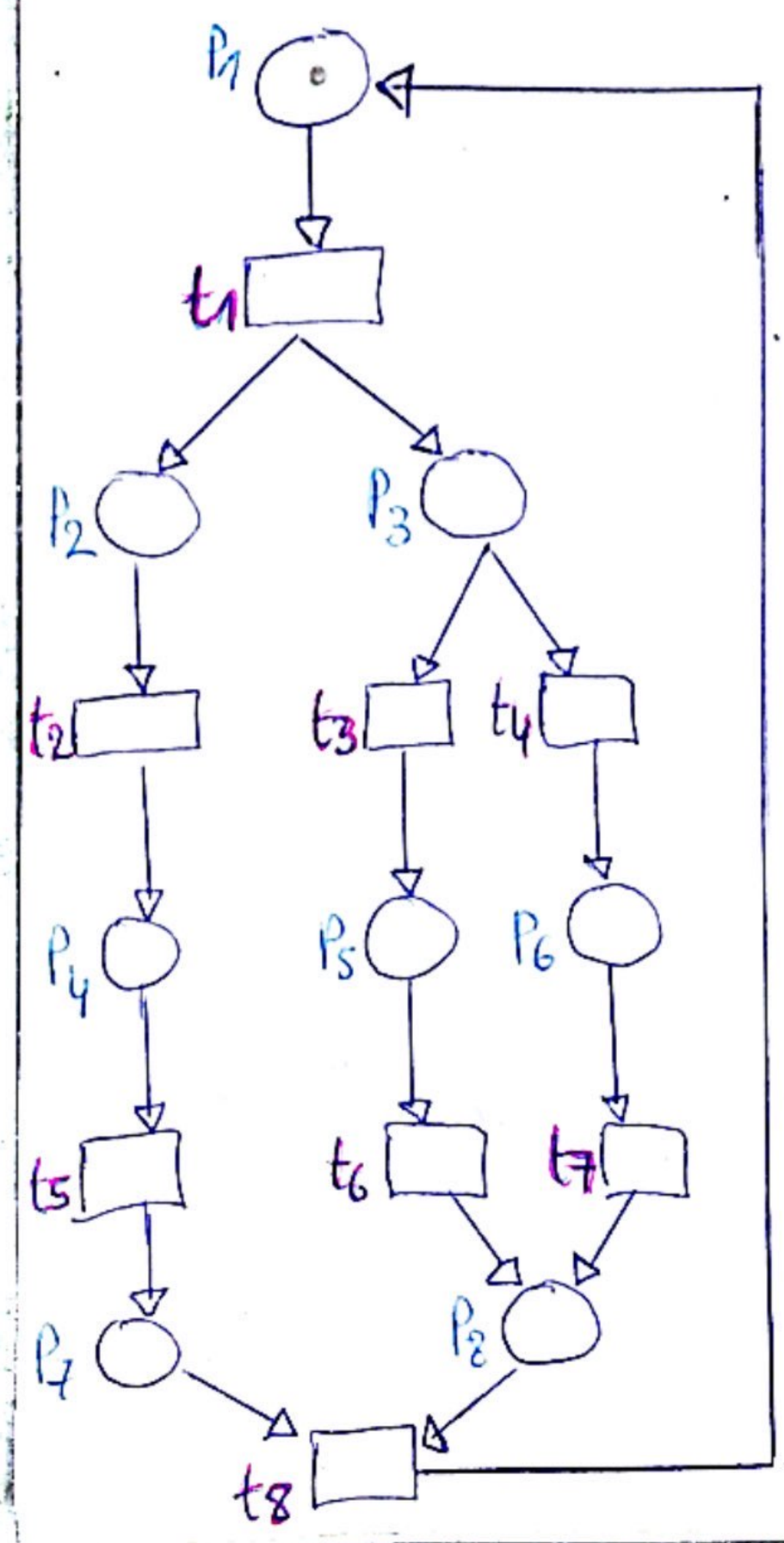
m

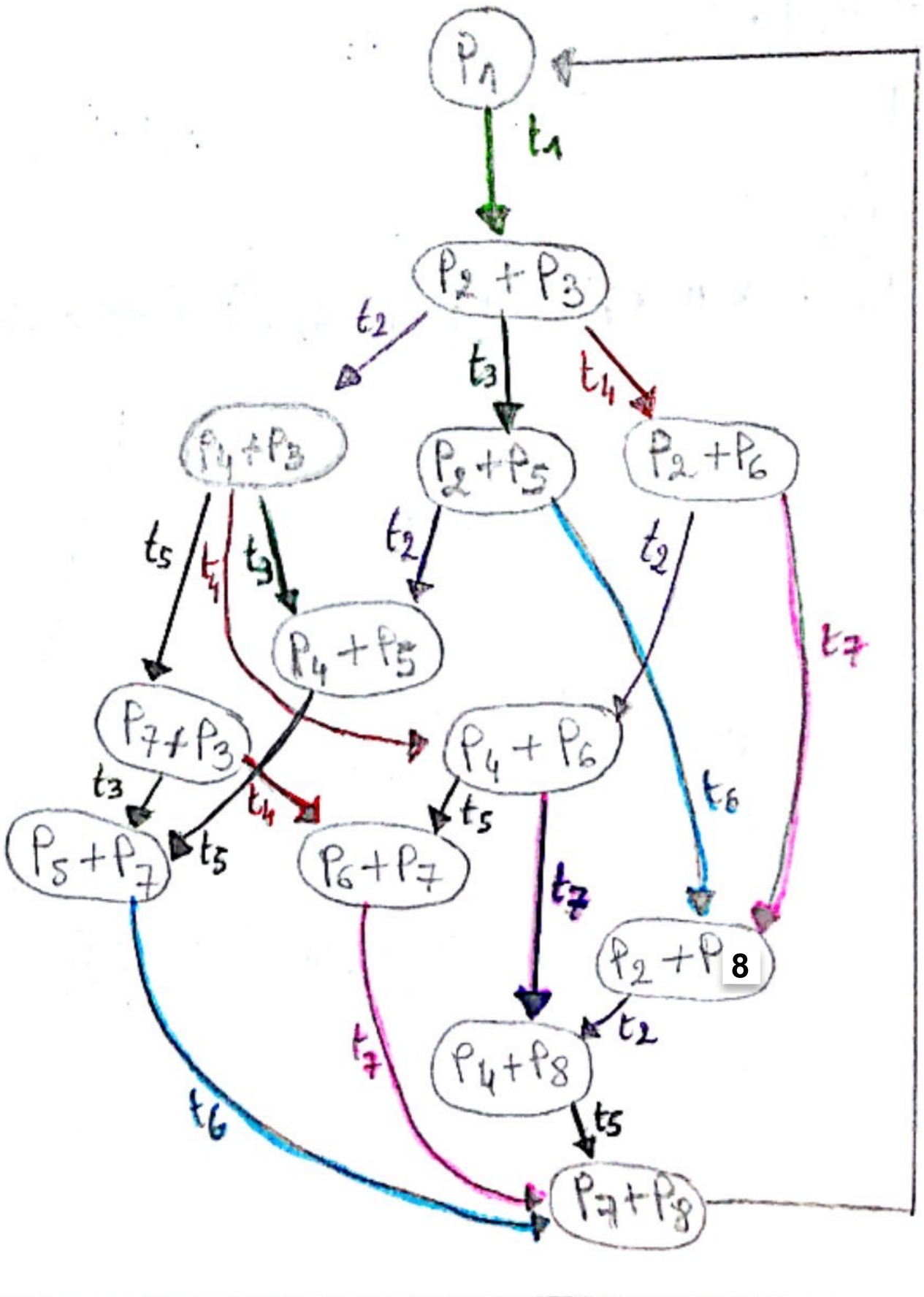
$\xrightarrow{t}$

après

1	P_1
0	P_2
1	P_3

m'



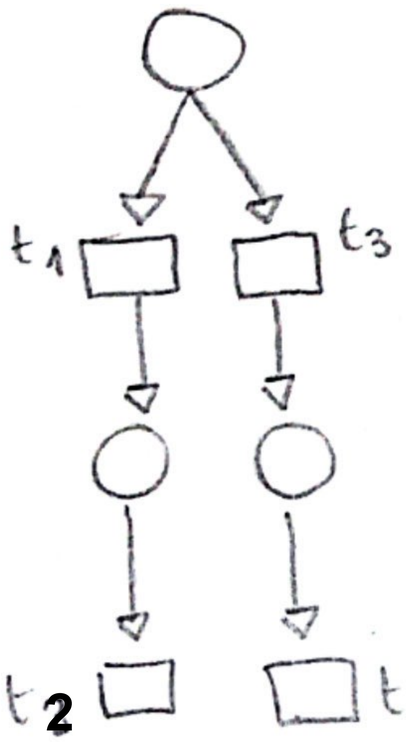


Scanned with CamScanner

- Une séquence $\sigma = t_1 \dots t_n \in T^*$ est franchissable à partir d'un marquage m ssi $\exists\, m_1, \dots, m_n$ tq $m \xrightarrow{t_1} m_1 \xrightarrow{t_2} m_2 \longrightarrow \dots \xrightarrow{t_n} m_n$

- On note $m \xrightarrow{\sigma}$

$L(R, m_0) = \{ \sigma \in T^* \mid m_0 \xrightarrow{\sigma} \}$



$(t_1 . t_2)^* . (t_3 . t_4)^*)^*$

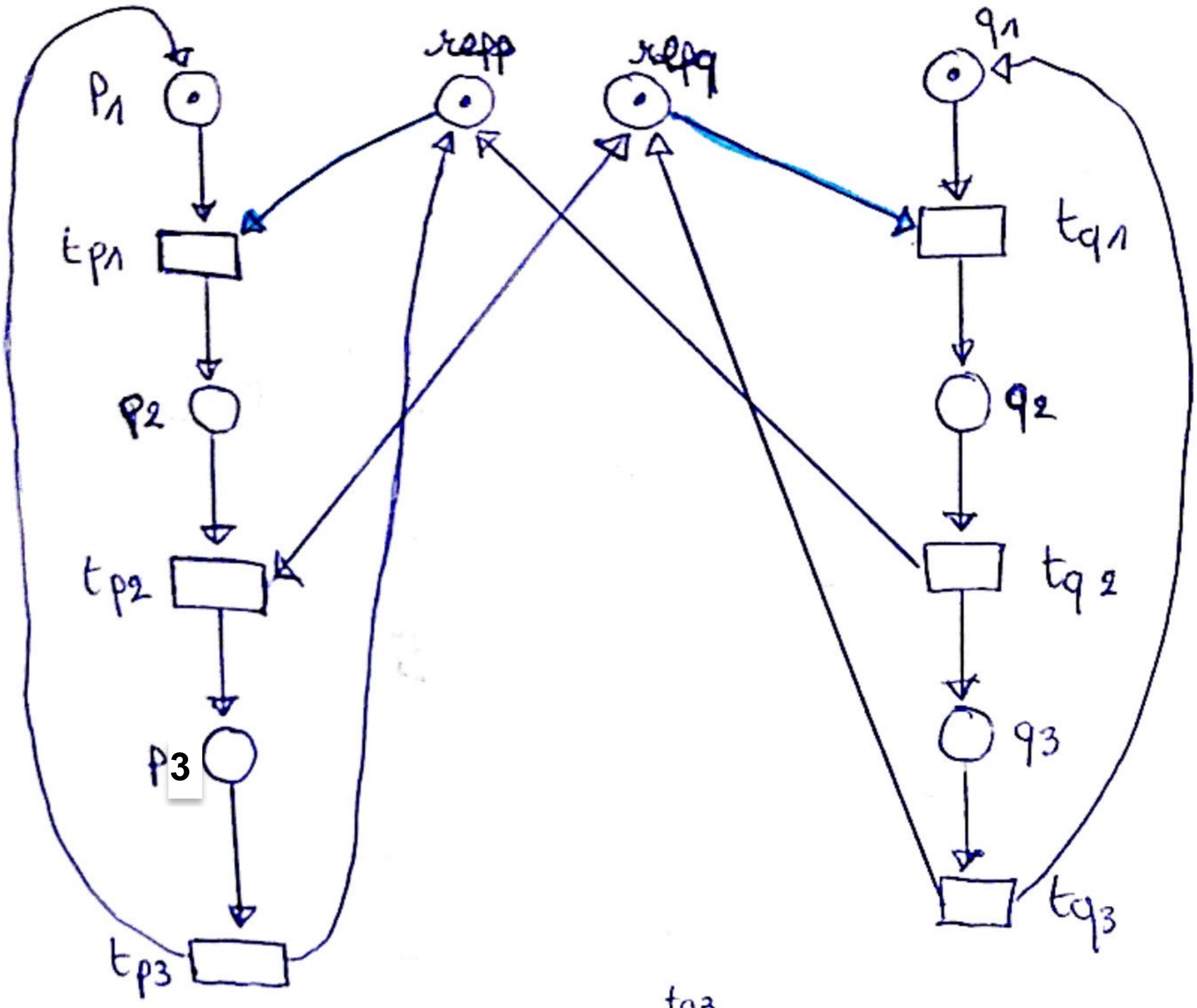
$Acc(R, m_0) = \{ m \mid \exists\, \sigma \in T^* : m_0 \xrightarrow{\sigma} m \}$

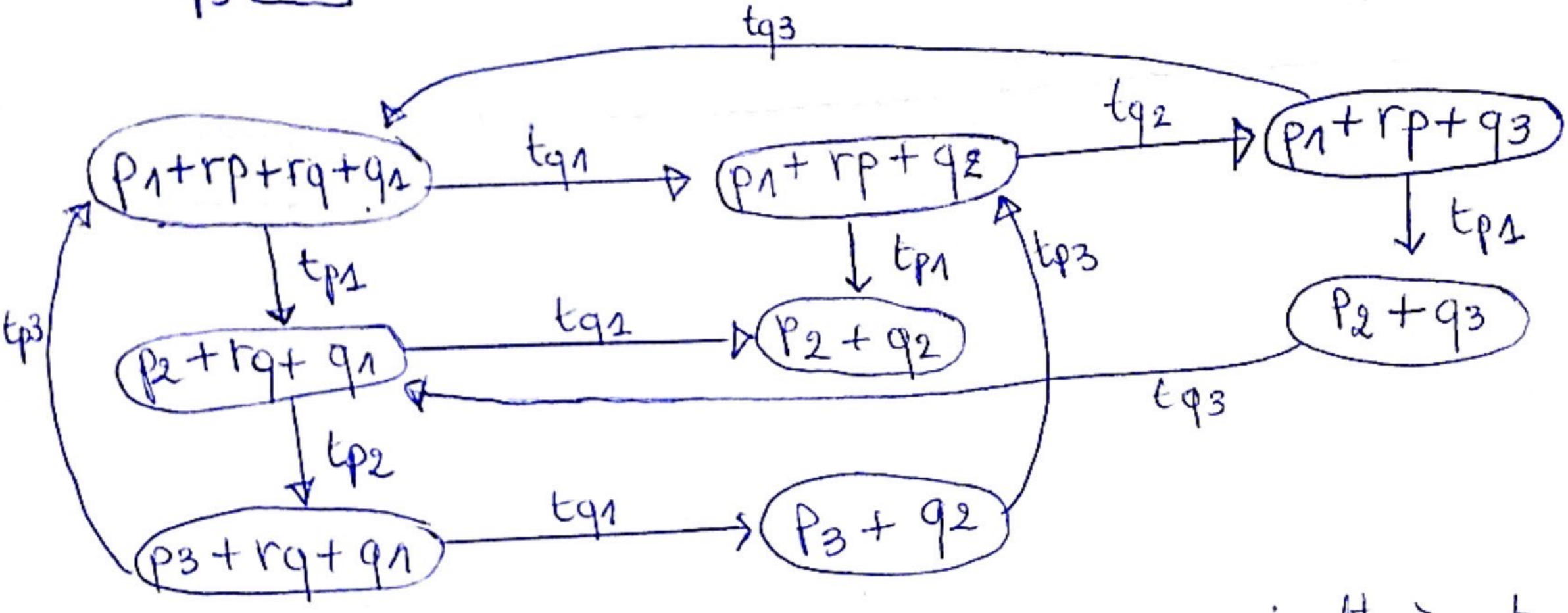
Graph des marquages accessibles d'un rdp (R, m_0)

c'est un syst. de transition $G < S', A, s_0, \rightarrow >$

- $S' = Acc(R, m_0)$
- $A = T$
- $s_0 = m_0$
- $\rightarrow \subseteq S \times A \times S'$ tq $(s, a, s') \in \rightarrow$ ssi $s \xrightarrow{a} s'$

Scanned with CamScanner



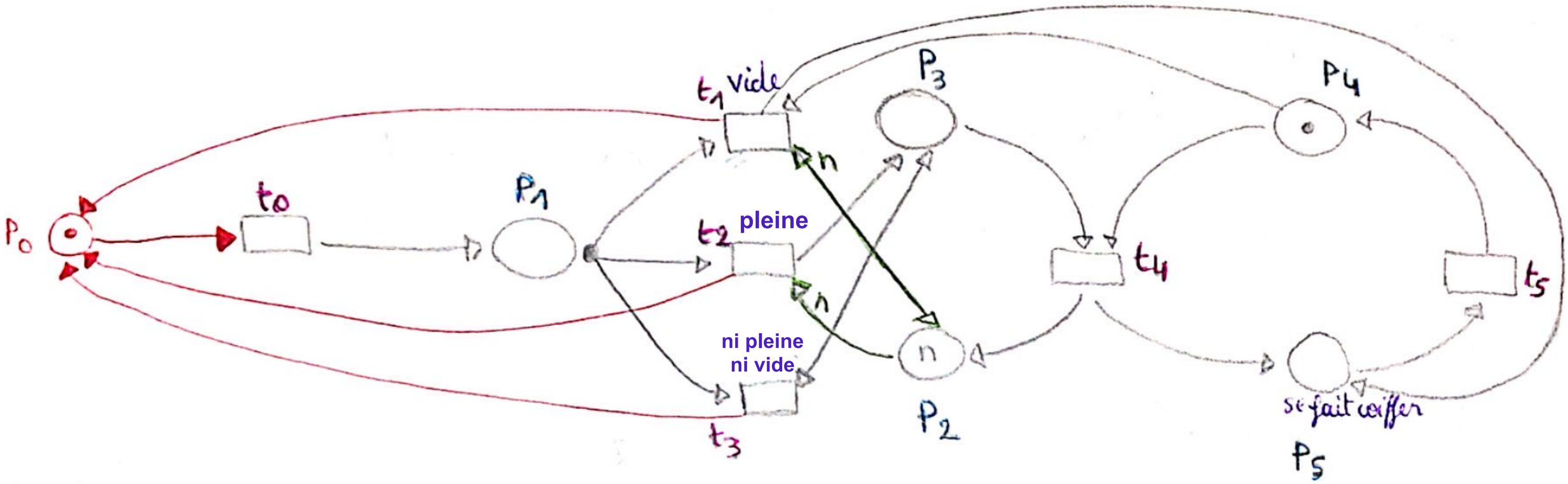


Jamais un jeton dans P3 et un jeton dans q3 simultanément.

Modélisation d'un salon de coiffure avec un rdp

- coiffeur peut être dans 2 états : libre ou occupé
- salle d'attente de capacité n
- quand un client arrive :
  - si coiffeur libre et salle d'attente vide ⇒ il se fait coiffer directement.
  - si salle d'attente pleine ⇒ il quitte le salon.
  - sinon il prend une salle d'attente.

Scanned with CamScanner



$n = 2$

$2P_2 + P_4$

$\downarrow t_0$

$P_1 + 2P_2 + P_4$

$\downarrow t_0$

$2P_1 + 2P_2 + P_4$

$\downarrow t_0$

$3P_1 + 2P_2 + P_4$

$\downarrow t_0$

Infini

Donc on ne peut créer un graphe de marquage.
Pour y remédier, on va faire un SAS.

Scanned with CamScanner

- Une place P d'un RDP (R, m_0) est dite bornée ssi :

$\exists\, k \in \mathbb{N}$ tq $\forall\, m \in Acc(R, m_0)$, $m(P) \leq k$, on dit que P est k-bornée

($Acc(R, m_0)$: ensemble des marquages ; k : nb de jetons)

- Un RDP(R, m_0) est borné ssi ttes ses places le sont. La borne du réseau est alors le max des bornes de ses places.
- Un RDP R est dit structurellement borné ssi $\forall$ le marquage m_0 (R, m_0) est bornés.

- Une transition t est quasi-vivante ssi $\exists\, m \in Acc(R, m_0)$ tq $m \xrightarrow{t}$
- (R, m_0) est quasi-vivante ssi ttes ses transitions le sont.

- Une transition t est dite vivante ssi $\forall\, m \in Acc(R, m_0)$, $\exists\, m' \in Acc(R, m)$ tq $m' \xrightarrow{t}$
- Un rdp (R, m_0) est dit vivant ssi ttes ses transitions le sont.

- Un rdp (R, m_0) est dit pseudo-vivant ssi il contient pas de marquage bloquant/mort càd : $\forall\, m \in Acc(R, m_0)$, $\exists\, t \in T$ tq $m \xrightarrow{t}$

quasi-vivant vs vivant
quasi-vivant vs pseudo-vivant
vivant vs pseudo-vivant

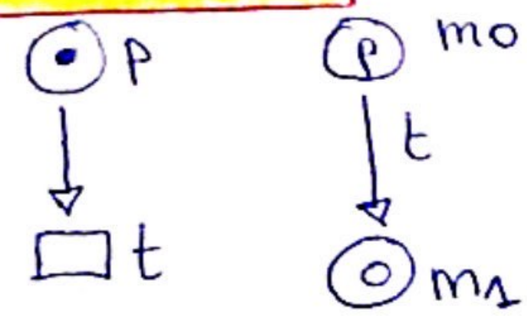
on doit ê capable de franchir dans le futur.

Scanned with CamScanner

quasi-vivant vs vivant

quasi-vivant ⇏ vivant

contre-exemple



m_1 n'est pas accessible à partir de la transition

vivant ⇒ quasi-vivant

Soit (R, m_0) un réseau vivant
Soit t une transition de R

t vivante ⇒ $\forall m \in Acc(R, m_0), \exists m \in Acc(R, m)$ tq $m' \xrightarrow{t}$, $m_0 \in Acc(R, m_0)$ ⇒ Obvious

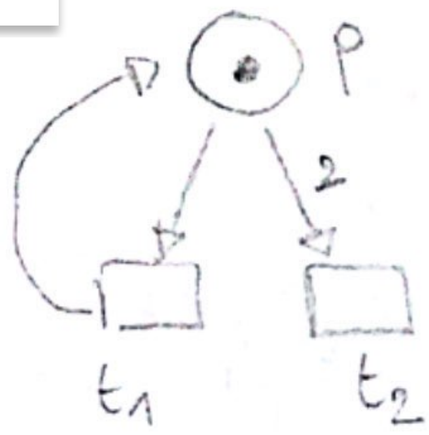
$\exists m' \in Acc(R, m_0)$ tq $m' \xrightarrow{t}$ ⇒ t est quasi-vivante

Pour tous les marquages appartenant à l'ensemble des marquages accessibles, à partir de l'état initial, Il existe un marquage m appartenant à l'ensemble des marquages accessibles tel que m' où la transition t est franchissable et m0 appartient à l'ensemble des marquages accessibles.

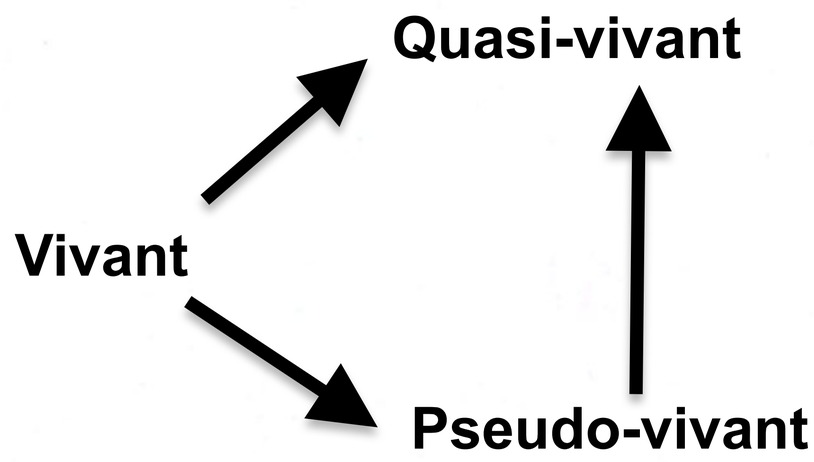
Il existe un marquage m' appartenant à l'ensemble des marquages accessibles où la transition t est franchissable

vivant vs pseudo-vivant

pseudo-vivant ⇏ vivant





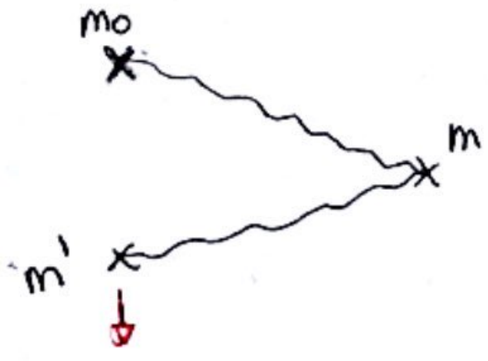


- Soit (R, m_0) un rdp.
- Supposons que (R, m_0) soit vivant.
- Montrons qu'il est forcément pseudo-vivant.
- Raisonnement que (R, m_0) soit un pseudo-vivant.

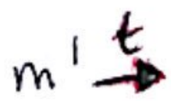
→ $\exists m^* \in Acc(R, m_0)$ tq $\forall t \in T, m^* \xrightarrow{t}$ est absurde car $\forall t \in T, \forall m \in Acc(R, m_0)$

→ $m' \in Acc(R, m)$ tq $m' \xrightarrow{t}$

→ pour m^* il devrait y avoir un marquage $m' \in Acc(R, m^*)$ tq $m' \xrightarrow{t}$

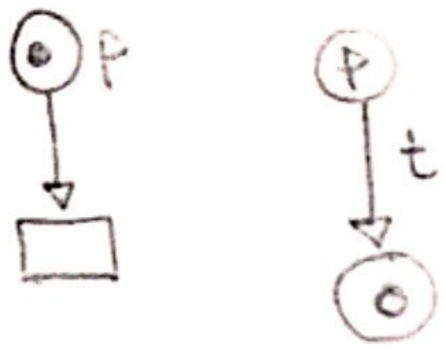




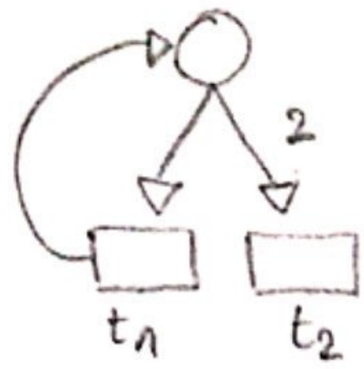


Scanned with CamScanner

quasi-vivant ⇏ pseudo-vivant

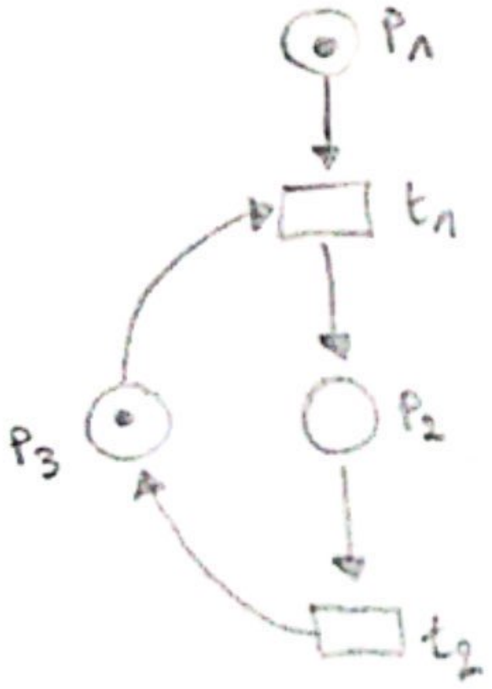


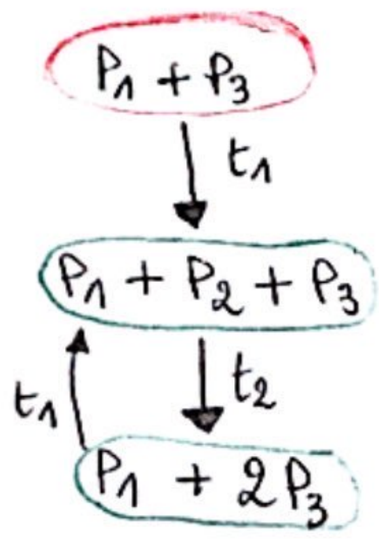
pseudo-vivant ⇒ quasi-vivant



- Un marquage m^* est dit état d'accueil dans (R, m_0) ssi $\forall m \in Acc(R, m_0), m^* \in Acc(R, m)$
- Si le marquage initial est un état d'accueil, on dit que le réseau est réversible.

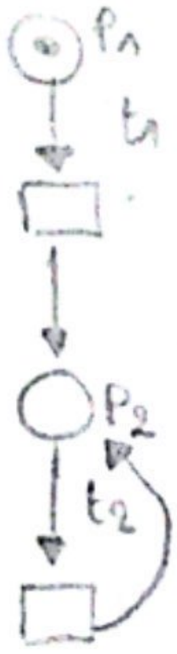
Ex: ① pseudo-vivant ⊕ quasi-vivant $\overset{?}{\Rightarrow}$ vivant FAUX
② réversible ⊕ quasi-vivant $\overset{?}{\Rightarrow}$ vivant VRAI

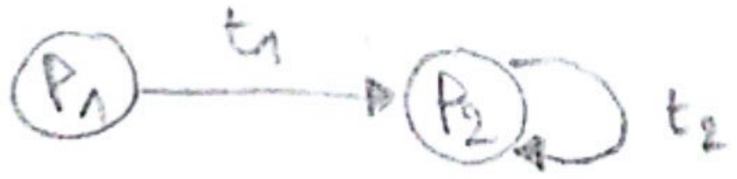




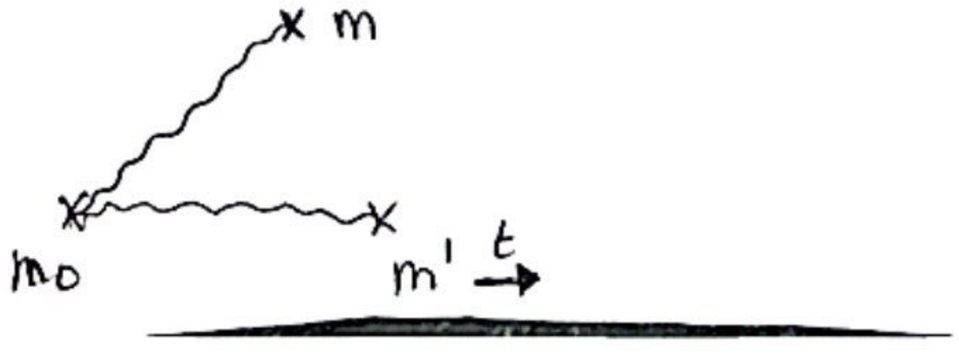
Pas état d'accueil
Etat d'accueil : je peux l'atteindre à partir de n'importe quel état

① N'existe pas de moyen de franchir t_1 par P_2 dans le futur donc n'est pas vivant.





② Soit (R, m_0) quasi-vivant est réversible, montrons qu'il est vivant.
Soit $t \in T$, soit $m \in Acc(R, m_0)$, montrons qu'il existe $m' \in Acc(R, m)$ où $m' \xrightarrow{t}$ est un état d'accueil, $\exists \sigma \in T^*$ tq $m \xrightarrow{\sigma} m$ et puisque t est quasi-vivante, $\exists \alpha \in T^*$, $m_0 \xrightarrow{\alpha} m'$ avec $m' \xrightarrow{t}$
Donc la séquence $\sigma.\alpha$ est franchissable à partir de m et mène vers m' où t est franchissable.
⇒ t est vivante ⇒ le rdp est vivant



Scanned with CamScanner

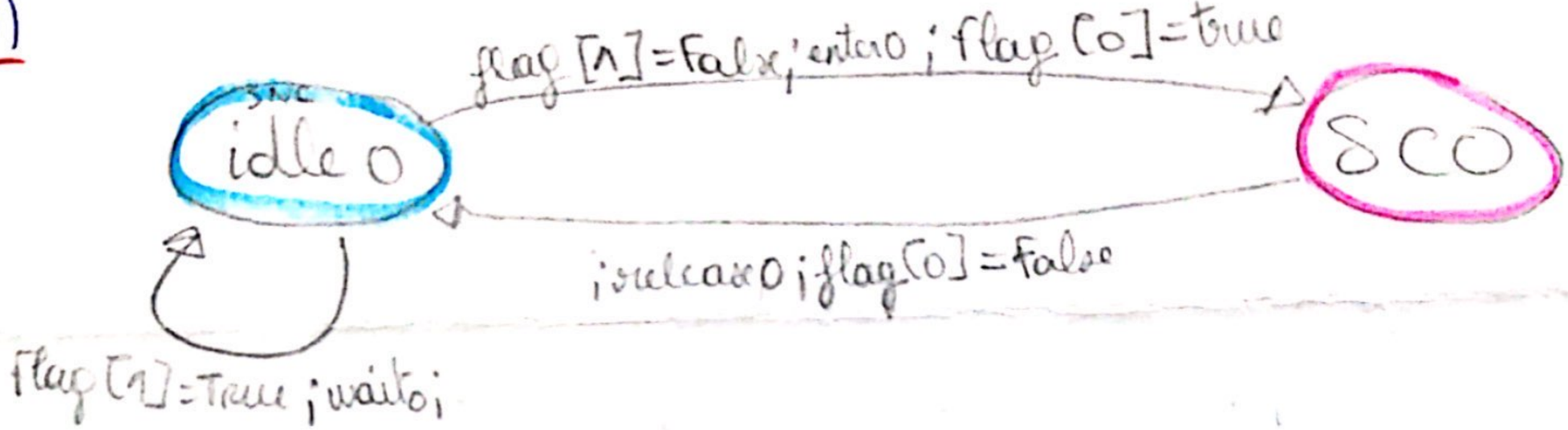
On considère le code suivant pour 2 proc. 27/10/2023

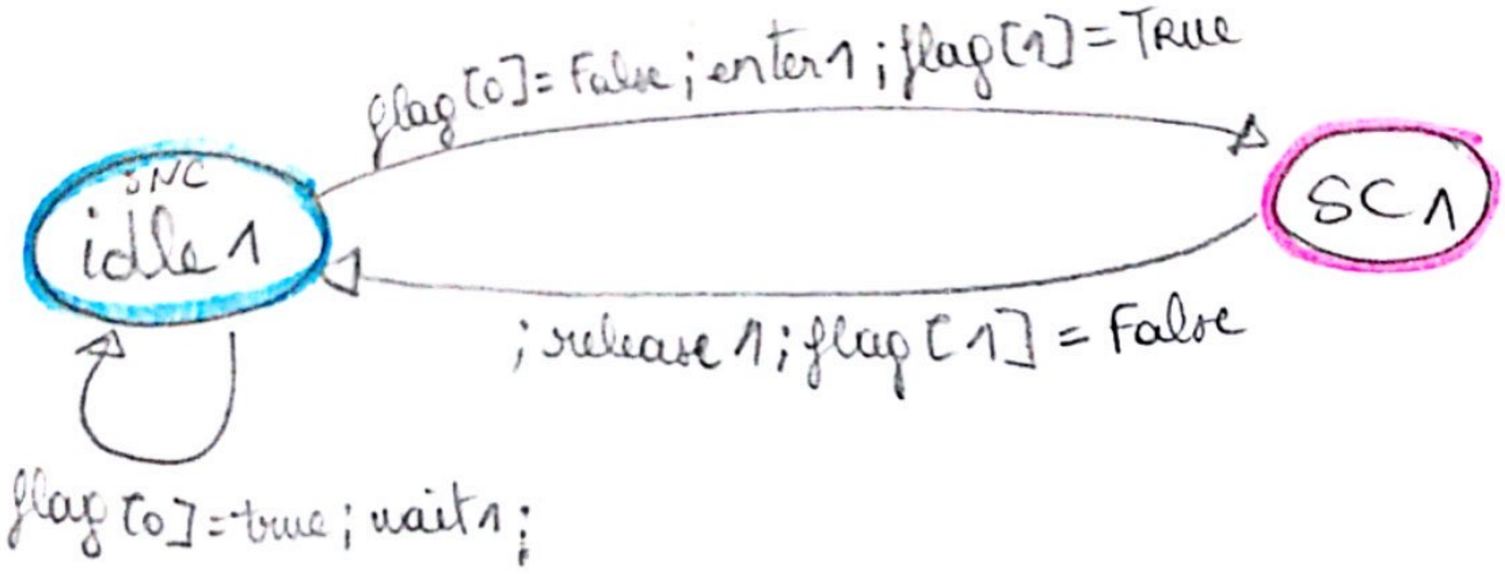
```
bool flag [2];          // tableau partagé, initialisé à false
Thread (i) {
    while (flag [1-i] == true);
    flag [i] = True
    section_ critique ();
    flag [i] = False
    section_non_critique ();
}
```

1) Modéliser le programme avec une machine à état
2) Donner le système de transition correspondant.

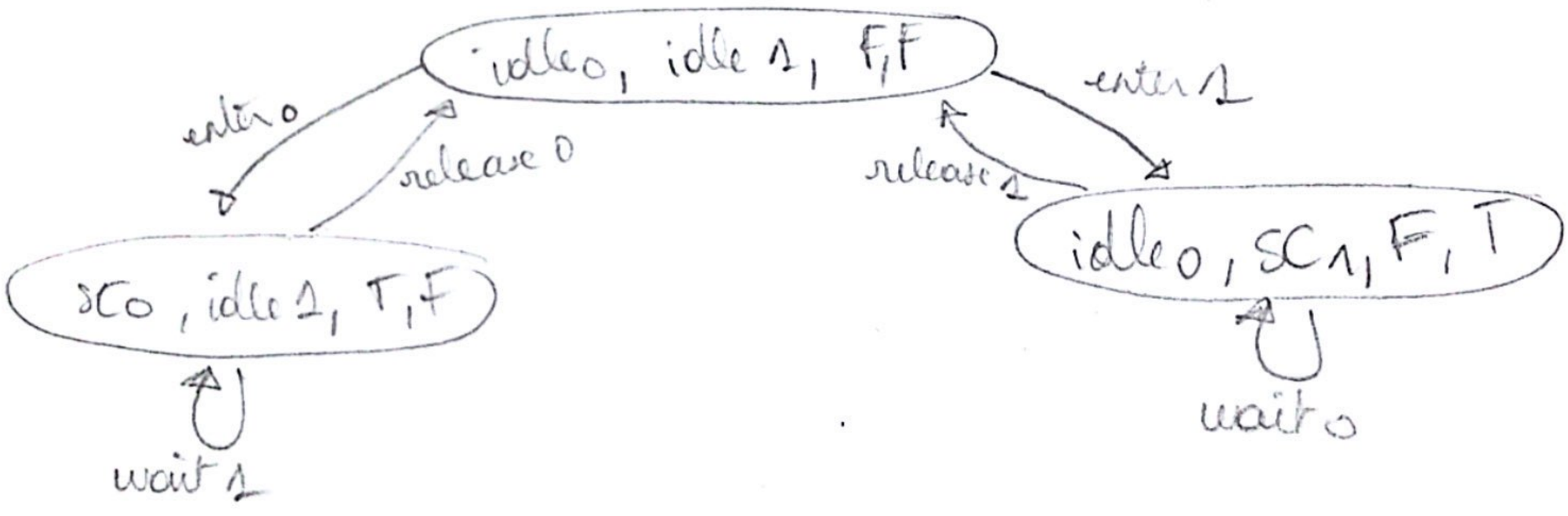
Scanned with CamScanner

1)





2)



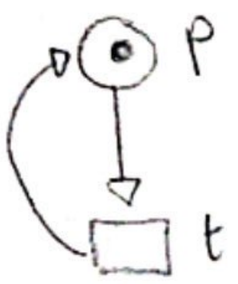
Scanned with CamScanner

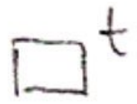
1) Dans chacun des cas suivants, donner un exemple de rdp qui satisfait les pptés indiquées, quand un tel réseau n'existe pas, expliquer pourquoi.

- un rdp vivant et pseudo-vivant
- un rdp quasi-vivant mais pas vivant
- un rdp vivant mais pas quasi-vivant
- un rdp non borné et non pseudo-vivant

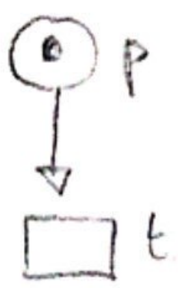
Solution :

1) a)



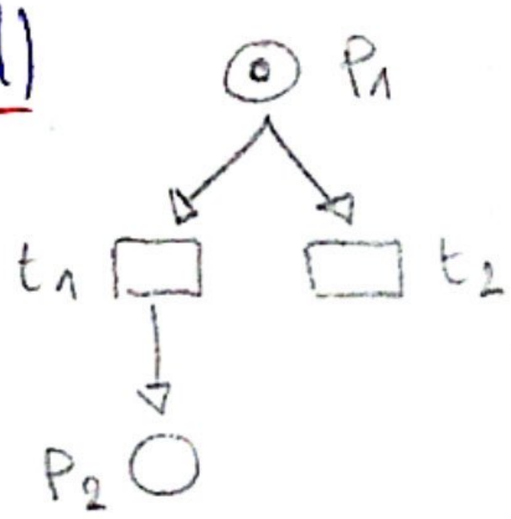


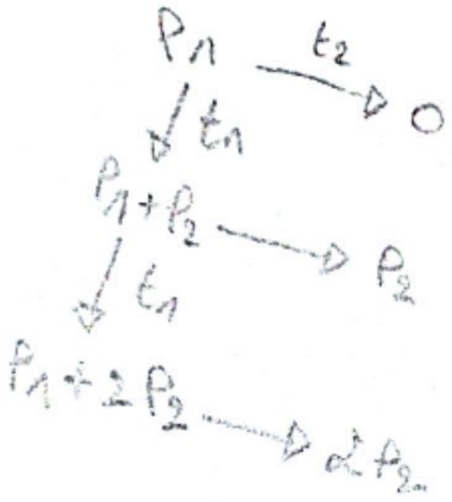
b)



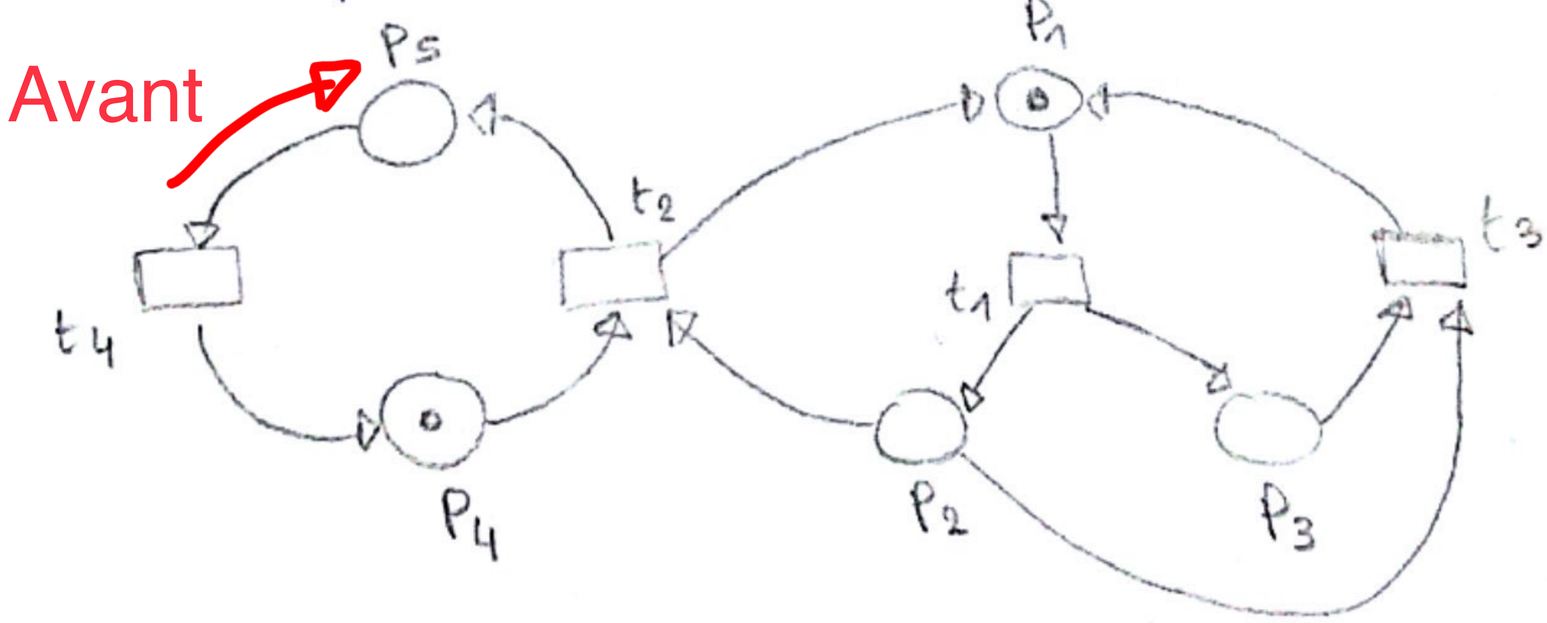
c) Non, car
vivant => quasi-vivant

d)





2) Considérer le rdp suivant



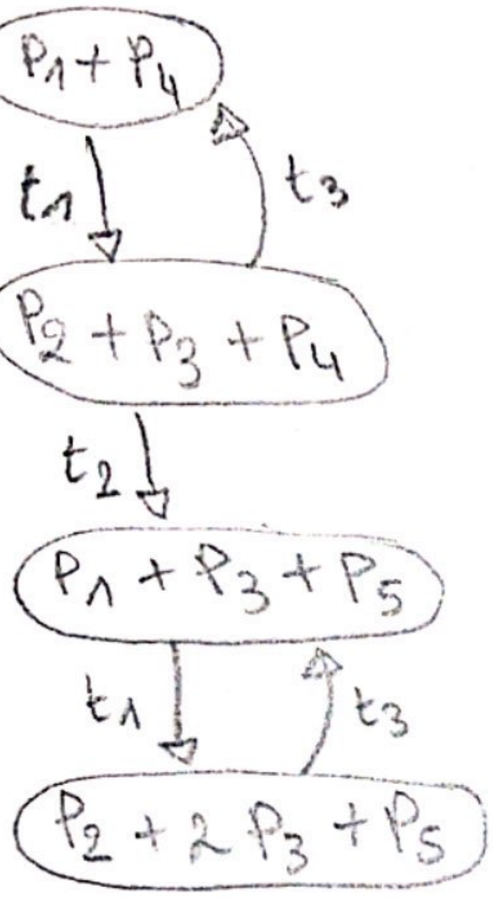
a) Construire le graphe des marquages

b) Est-ce un rdp borné ? quasi-vivant ? vivant ? pseudo-vivant ?
V F F V

c) Est-il possible de franchir un jour t_4 ? sinon, modifier le réseau aussi peu que possible pour rendre t_4 franchissable.

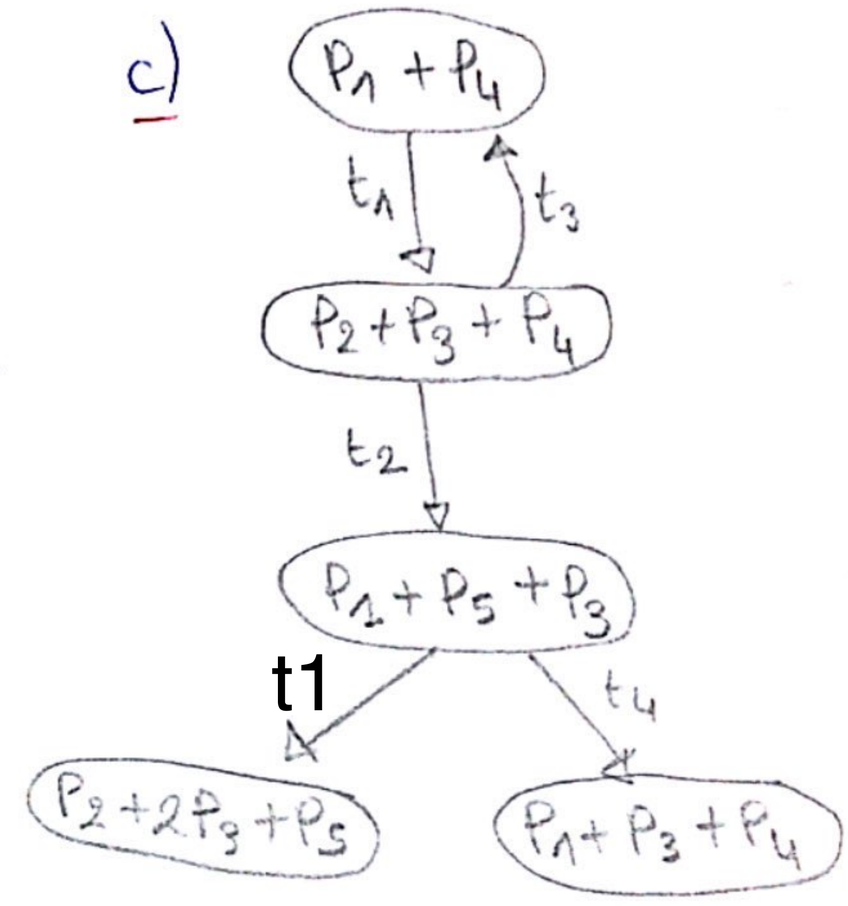
Scanned with CamScanner

a)



b) Oui, borné
Non, quasi-vivant
Non, vivant
Oui, pseudo-vivant

c)





changer sens de la flèche

Scanned with CamScanner